

Task 2B: Code Metrics Analysis

Tools Used

- PMD
 - Static code analysis for Java
 - Provides metrics related to:
 - Complexity (Cyclomatic, Cognitive, NPath)
 - Object-Oriented design (God Class, Data Class, Coupling)
 - Code smells (Exception as Flow Control, Excessive Imports, Long Parameter Lists)

These tools provide a good review of maintainability, readability, and architectural quality.

Top Architectural Violators by Metric

1. Cognitive Complexity

Definition: Measures how difficult code is to understand, considering nesting, control flow, and logical breaks.

1. DatabaseInstaller.java – 400
2. HTMLSanitizer.java – 160
3. PageServlet.java – 133
4. WeblogPageRequest.java – 71
5. MenuHelper.java – 65
6. WeblogRequestMapper.java – 64
7. CommentServlet.java – 64
8. ThemeManagerImpl.java – 60
9. UISecurityInterceptor.java – 55
10. MediaFileAdd.java – 52

Implications:

- Extremely high cognitive complexity indicates poor readability
- Increases error probability
- Makes safe modification in the codebase difficult

2. Coupling Between Objects (CBO) — CK Metric

Definition: Counts the number of classes a class is coupled with.

1. JPAWeblogManagerImpl.java – 53
2. JPAWeblogEntryManagerImpl.java – 47

3. SiteModel.java – 35
4. JPAMediaFileManagerImpl.java – 32
5. Weblog.java – 30
6. MediaCollection.java – 29
7. TestUtils.java – 29
8. WeblogEntry.java – 28
9. EntryCollection.java – 26
10. ThemeManagerImpl.java – 25

Implications:

- **High coupling** reduces modularity and flexibility
- Small changes may affect many areas across the system
- Unit testing becomes harder

3. Cyclomatic Complexity

Definition: Measures the number of independent execution paths in the code.

1. DatabaseInstaller.java – 400
2. JPAWeblogEntryManagerImpl.java – 201
3. WeblogEntry.java – 156
4. Weblog.java – 153
5. Utilities.java – 146
6. JPAWeblogManagerImpl.java – 112
7. JPAMediaFileManagerImpl.java – 111
8. PageServlet.java – 108
9. MailUtil.java – 103
10. MediaCollection.java – 99

Implications:

- High defect probability
- Increased testing effort
- Logic is difficult to reason about

4. Data Class Smell

Definition: Classes that mainly store data with little or no behavior.

1. Subscription.java – 30
2. SearchOperation.java – 30
3. TaskLock.java – 29
4. SQLScriptRunner.java – 28

5. PingQueueEntry.java – 28
6. Setup.java – 28
7. MediaFileEdit.java – 28
8. Templates.java – 28
9. SharedThemeTemplateRendition.java – 27
10. WeblogBookmark.java – 27

Implications:

- Encourages procedural programming
- Breaks encapsulation
- Business logic spreads across the system

5. Exception as Flow Control

Definition: Using exceptions for normal control flow instead of error handling.

Here the numbers are the line location in the respective files.

1. MediaCollection.java – 453
2. MetaWeblogAPIHandler.java – 393
3. EntryCollection.java – 289
4. DatabaseInstaller.java – 221
5. CommentServlet.java – 206
6. BloggerAPIHandler.java – 198
7. PageServlet.java – 166
8. SearchServlet.java – 147
9. TrackbackServlet.java – 135
10. ThemeResourceLoader.java – 112

Implications:

- Decreases performance
- Hides real error conditions
- Makes debugging harder

6. God Class - (CK-related Design Smell)

Definition:

A class that centralizes too much behavior and responsibility. The God Class detection is based on the **WMC (Weighted Methods per Class)** score

1. WeblogEntry.java – 156
2. Weblog.java – 153
3. Utilities.java – 146
4. DatabaseInstaller.java – 134
5. PageServlet.java – 108
6. MailUtil.java – 103
7. MediaFile.java – 87
8. HTMLSanitizer.java – 83
9. RollerAtomHandler.java – 83
10. DateUtil.java – 79

Implications:

- Violates Single Responsibility Principle
- Difficult to extend or reuse
- High risk during maintenance

Overall Observations

- The project exhibits high complexity and strong coupling
- Multiple God Classes and low-cohesion between classes
- Excessive use of exceptions and large methods
- Several core components need refactoring

Conclusion

1. The code metrics analysis reveals that Apache Roller's initial state has significant architectural issues and a very high technical debt.
2. However, the metrics also provide clear guidance on where refactoring efforts should begin. The maintainability, readability, and scalability of the system can be significantly improved. By prioritizing:
 - High complexity classes
 - Highly coupled components
 - God classes and data classes